

```

/* =====
Author    : TSI_Sabit
Region    : Bangladesh
Bangladesh University of Business and Technology
===== */

#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define int long long int
#define pb push_back
#define ppb pop_back
#define mp make_pair
#define fi first
#define se second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define yes cout << "YES" << endl;
#define no cout << "NO" << endl;
#define nn "\n"
#define sz(x) (int)(x).size()
#define gcd(a, b) __gcd(a, b)
#define lcm(a, b) ((a) / gcd(a, b) * (b))
#define ms(arr, v) memset(arr, v, sizeof(arr))
#define rep(i, a, b) for (int i = a; i < b; i++)
#define rrep(i, a, b) for (int i = a; i >= b; i--)
#define rep0(i, n) for (int i = 0; i < n; i++)
#define rep1(i, n) for (int i = 1; i <= n; i++)
#define read(x) int x; cin >> x;
#define readv(v, n) vector<int> v(n); for (auto &i : v) cin >> i;
#define setbit(x) __builtin_popcountll(x)
#define parity(x) __builtin_parityll(x)
#define clz(x) __builtin_clzll(x)
#define ctz(x) __builtin_ctzll(x)
#define unq(v) v.resize(distance(v.begin(), unique(v.begin(), v.end())));

#define pii pair<int, int>
#define pll pair<ll, ll>
#define vi vector<int>
#define vll vector<ll>
#define vpii vector<pii>
#define vpll vector<pll>

const int MOD = 1e9 + 7;
const int INF = 1e9;
const ll LINF = 1e18;
const double PI = acos(-1);

void fastIO() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);
}

ll power(ll a, ll b, ll mod = MOD) {
    ll res = 1;
    while (b > 0) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

ll modInv(ll a, ll mod = MOD) {
    return power(a, mod - 2, mod);
}

int binaryAsDecimal(int num) {

```

```

    int res = 0, base = 1;
    while (num > 0) {
        res += (num % 2) * base;
        base *= 10;
        num /= 2;
    }
    return res;
}

bool isPowerOfTwo(ll n) {
    return (n > 0 && (n & (n - 1)) == 0);
}

ll binarySearch(ll low, ll high, function<bool(ll)> check) {
    while (low < high) {
        ll mid = (low + high) / 2;
        if (check(mid)) high = mid;
        else low = mid + 1;
    }
    return low;
}

void vectorPrint(vector<int>& v) {
    for (auto val : v) cout << val << ' ';
    cout << nn;
}

bool isSorted(const vector<int>& arr) {
    for (int i = 1; i < arr.size(); ++i) {
        if (arr[i] < arr[i - 1]) return false;
    }
    return true;
}

bool isSortedDescending(const vector<int>& arr) {
    for (int i = 1; i < (int)arr.size(); ++i) {
        if (arr[i] > arr[i - 1]) return false;
    }
    return true;
}

string intToBinary(int n) {
    if (n == 0) return "0";
    string res = "";
    while (n > 0) {
        res += (n % 2) + '0';
        n /= 2;
    }
    reverse(res.begin(), res.end());
    return res;
}

vector<int> prefixSum(const vector<int>& v) {
    int n = v.size();
    vector<int> pre(n);
    pre[0] = v[0];
    for (int i = 1; i < n; i++) {
        pre[i] = pre[i - 1] + v[i];
    }
    return pre;
}

vector<int> suffixSum(const vector<int>& v) {
    int n = v.size();
    vector<int> suf(n);
    suf[n - 1] = v[n - 1];
    for (int i = n - 2; i >= 0; i--) {
        suf[i] = suf[i + 1] + v[i];
    }
    return suf;
}

```

```

int arrayGCD(const vector<int>& v) {
    int res = v[0];
    for (int i = 1; i < v.size(); i++) {
        res = gcd(res, v[i]);
        if (res == 1) break;
    }
    return res;
}

int arrayLCM(const vector<int>& v) {
    int res = v[0];
    for (int i = 1; i < v.size(); i++) {
        res = lcm(res, v[i]);
    }
    return res;
}

map<int, int> frequencyCount(const vector<int>& v) {
    map<int, int> freq;
    for (auto val : v) {
        freq[val]++;
    }
    return freq;
}

map<char, int> charFrequency(const string& s) {
    map<char, int> freq;
    for (char c : s) {
        freq[c]++;
    }
    return freq;
}

const int MAXN = 1000000;
int spf[MAXN + 1];

void sieveSPF() {
    for (int i = 1; i <= MAXN; i++) spf[i] = i;
    for (int i = 2; i * i <= MAXN; i++) {
        if (spf[i] == i) {
            for (int j = i * i; j <= MAXN; j += i) {
                if (spf[j] == j) spf[j] = i;
            }
        }
    }
}

vector<int> getFactorization(int x) {
    vector<int> res;
    while (x != 1) {
        res.push_back(spf[x]);
        x /= spf[x];
    }
    return res;
}

int sumOfDivisors(int n) {
    int ans = 1;
    while (n > 1) {
        int p = spf[n];
        int cnt = 0;
        while (n % p == 0) {
            n /= p;
            cnt++;
        }
        int term = 1, power = 1;
        for (int i = 0; i < cnt; i++) {
            power *= p;
            term += power;
        }
        ans *= term;
    }
}

```

```

    }
    return ans;
}

int numOfDivisors(int n) {
    int ans = 1;
    while (n > 1) {
        int p = spf[n];
        int cnt = 0;
        while (n % p == 0) {
            n /= p;
            cnt++;
        }
        ans *= (cnt + 1);
    }
    return ans;
}

int productOfDivisors(int n) {
    int d = numOfDivisors(n);
    return power(n % MOD, d / 2, MOD);
}

int mostFrequentElement(const vector<int>& v) {
    map<int, int> freq;
    int maxFreq = 0, mostFreq = v[0];
    for (int val : v) {
        freq[val]++;
        if (freq[val] > maxFreq) {
            maxFreq = freq[val];
            mostFreq = val;
        }
    }
    return mostFreq;
}

const int N = 1e6 + 5;
bool is_prime[N];
void sieve(){
    fill(is_prime, is_prime + N, true);
    is_prime[0] = is_prime[1] = false;
    for(int i = 2; i * i < N; i++){
        if(is_prime[i]){
            for(int j = i * i; j < N; j += i){
                is_prime[j] = false;
            }
        }
    }
}

ll factorial(ll n) {
    if (n == 0 || n == 1) return 1;
    return n * factorial(n - 1);
}

ll fact[MAXN];

void computeFactorials(int n, ll mod = MOD) {
    fact[0] = 1;
    for (int i = 1; i <= n; i++) {
        fact[i] = (fact[i - 1] * i) % mod;
    }
}

vector<int> prefixMinimum(const vector<int>& a) {
    int n = a.size();
    vector<int> pre(n);
    pre[0] = a[0];
    for (int i = 1; i < n; i++) {
        pre[i] = min(pre[i - 1], a[i]);
    }
    return pre;
}

```

```

}

vector<int> suffixMinimum(const vector<int>& a) {
    int n = a.size();
    vector<int> suf(n);
    suf[n - 1] = a[n - 1];
    for (int i = n - 2; i >= 0; i--) {
        suf[i] = min(suf[i + 1], a[i]);
    }
    return suf;
}

vector<int> prefixMaximum(const vector<int>& a) {
    int n = a.size();
    vector<int> pre(n);
    pre[0] = a[0];
    for (int i = 1; i < n; i++) {
        pre[i] = max(pre[i - 1], a[i]);
    }
    return pre;
}

vector<int> suffixMaximum(const vector<int>& a) {
    int n = a.size();
    vector<int> suf(n);
    suf[n - 1] = a[n - 1];
    for (int i = n - 2; i >= 0; i--) {
        suf[i] = max(suf[i + 1], a[i]);
    }
    return suf;
}

int phi[N];
void computeTotient() {
    for(int i = 0; i < N; i++) phi[i] = i;
    for(int i = 2; i < N; i++) {
        if(phi[i] == i) {
            for(int j = i; j < N; j += i) {
                phi[j] -= phi[j] / i;
            }
        }
    }
}

int nthFibonacci(int n) {
    if (n == 1 || n == 2) return 1;
    int a = 1, b = 1, fib = 0;
    for (int i = 3; i <= n; i++) {
        fib = (a + b) % MOD;
        a = b;
        b = fib;
    }
    return fib;
}

char mostFrequentCharacter(const string& s) {
    map<char, int> freq;
    for(char c : s)
        freq[c]++;
    char ans = s[0];
    int mx = 0;
    for(pair<char, int> p : freq) {
        if(p.second > mx) {
            mx = p.second;
            ans = p.first;
        }
    }
    return ans;
}

int LCSLength(string a, string b) {
    int n = a.size();
    int m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1,
0));

```

```

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                if (a[i - 1] == b[j - 1]) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                }
                else {
                    dp[i][j] = max(dp[i - 1][j], dp[i][j -
1]);
                }
            }
        }
        return dp[n][m];
    }

    string LCS(string a, string b) {
        int n = a.size();
        int m = b.size();
        vector<vector<int>> dp(n + 1, vector<int>(m + 1,
0));
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                if (a[i - 1] == b[j - 1]) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                }
                else {
                    dp[i][j] = max(dp[i - 1][j], dp[i][j -
1]);
                }
            }
        }
        string ans = "";
        int i = n, j = m;
        while (i > 0 && j > 0) {
            if (a[i - 1] == b[j - 1]) {
                ans += a[i - 1];
                i--;
                j--;
            }
            else if (dp[i - 1][j] > dp[i][j - 1]) {
                i--;
            }
            else {
                j--;
            }
        }
        reverse(all(ans));
        return ans;
    }

    int LongestCommonSubstring(string a, string b) {
        int n = a.size();
        int m = b.size();
        vector<vector<int>> dp(n + 1, vector<int>(m + 1,
0));
        int ans = 0;
        for(int i = 1; i <= n; i++) {
            for(int j = 1; j <= m; j++) {
                if(a[i - 1] == b[j - 1]) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                    ans = max(ans, dp[i][j]);
                }
            }
        }
        return ans;
    }

    int EditDistance(string a, string b) {
        int n = a.size();
        int m = b.size();
        vector<vector<int>> dp(n + 1, vector<int>(m + 1));
        for(int i = 0; i <= n; i++) dp[i][0] = i;
        for(int j = 0; j <= m; j++) dp[0][j] = j;
        for(int i = 1; i <= n; i++) {
            for(int j = 1; j <= m; j++) {
                if(a[i - 1] == b[j - 1])

```

```

        dp[i][j] = dp[i - 1][j - 1];
    else
        dp[i][j] = 1 + min({
            dp[i - 1][j],
            dp[i][j - 1],
            dp[i - 1][j - 1]
        });
    }
}
return dp[n][m];
}
vector<int> buildLPS(string pat) {
    int n = pat.size();
    vector<int> lps(n);
    int j = 0;
    for(int i = 1; i < n; i++) {
        while(j > 0 && pat[i] != pat[j])
            j = lps[j - 1];
        if(pat[i] == pat[j])
            j++;
        lps[i] = j;
    }
    return lps;
}
vector<int> KMP(string text, string pat) {
    vector<int> lps = buildLPS(pat);
    vector<int> pos;
    int j = 0;
    for(int i = 0; i < text.size(); i++) {
        while(j > 0 && text[i] != pat[j])
            j = lps[j - 1];
        if(text[i] == pat[j])
            j++;
        if(j == pat.size()) {
            pos.pb(i - pat.size() + 1);
            j = lps[j - 1];
        }
    }
    return pos;
}
vector<int> ZFunction(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for(int i = 1; i < n; i++) {
        if(i <= r)
            z[i] = min(r - i + 1, z[i - l]);
        while(i + z[i] < n &&
            s[z[i]] == s[i + z[i]])
            z[i]++;
        if(i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }
    return z;
}
struct DSU {
    vector<int> parent, sz;
    DSU(int n) {
        parent.resize(n + 1);
        sz.assign(n + 1, 1);
        for(int i = 0; i <= n; i++)
            parent[i] = i;
    }
    int find(int v) {
        if(parent[v] == v)
            return v;
        return parent[v] =
            find(parent[v]);
    }
    void unite(int a, int b) {

```

```

        a = find(a);
        b = find(b);
        if(a != b) {
            if(sz[a] < sz[b])
                swap(a, b);
            parent[b] = a;
            sz[a] += sz[b];
        }
    }
};
vector<long long> dijkstra(int n,
    vector<vector<pair<int, int>>> &adj,
    int src)
{
    vector<long long> dist(n + 1, LINF);
    priority_queue<pair<long long, int>,
        vector<pair<long long, int>>,
        greater<pair<long long, int>>> pq;
    dist[src] = 0;
    pq.push(make_pair(0LL, src));
    while (!pq.empty()) {
        long long d = pq.top().first;
        int u = pq.top().second;
        pq.pop();
        if (d > dist[u])
            continue;
        for (int i = 0; i < (int)adj[u].size(); i++) {
            int v = adj[u][i].first;
            int w = adj[u][i].second;
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pq.push(make_pair(dist[v], v));
            }
        }
    }
    return dist;
}
void Solve() {
}
int32_t main() {
    fastIO();
#ifdef LOCAL
    auto start =
        chrono::high_resolution_clock::now();
#endif
    int t;
    cin >> t;
    while (t--) {
        Solve();
    }
#ifdef LOCAL
    auto end = chrono::high_resolution_clock::now();
    cerr << "Time: " <<
        chrono::duration_cast<chrono::milliseconds>(end -
            start).count() << " ms\n";
#endif
    return 0;
}

```